Pseudocode Convention

1. Comments begin with // and continue until end of line
2. Blocks are indicated with matching braces { }
3. Compound statement (collection of simple statements) can be represented as a block
4. Body of procedure → block
5. Statements are delimited by ;
3. An identifier begins with a letter. Data types of variables are not explicitly declared. Whether a variable is global or local ~~can be declared~~ PC evident from context. Datatypes are char, integer, float and boolean. Compound datatypes are framed with records.
11. Assignment of values to vars is done using assignment statement. <variable> := <expression>;
Ex: A := A+1;
5. There are 2 boolean values — true & false
Logical operators and, or and not
Relational operators >, ≥, <, ≤, =, ≠
6. Elements of a multi-dimensional array are accessed using [and]. For ex, A is a 2-dimensional array, i & j are its 2 indices, (i,j)th element is denoted A[i, j]. Array index starts at 0
7. looping : for, while, repeat until

while <condition> do
{ ... }

for variable := value1 to value2 step step do
{ }

repeat
{ ... }
until <condition>

case {
: condition1 : stmt1
:
: conditionn : stmtn
: else : statementn+1
}

break; return;

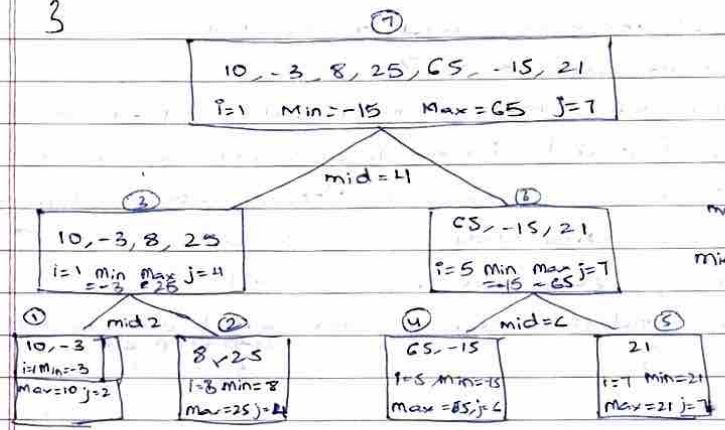
8. if <condition> then <statement>
if <condition> then <statement1> else <statement2>

9. Input & output — read & write. No format is done to specify size of input & output quantities.

10. One type of procedure:- Algorithm. Algo consists of heading & body:- Heading Algorithm Name(<parameter list>)
Simple vars or procedures are passed by value. Arrays & records are passed by reference. Array/Record name is treated as pointer to specific data type.
Algo may or may not return a value.

MinMax Algorithm

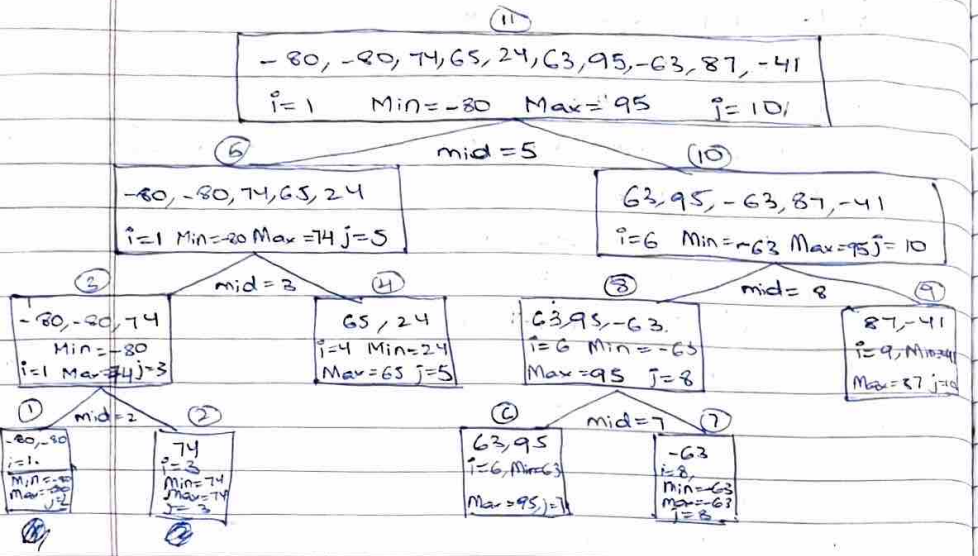
```
Algorithm DAndC (P)
{ if small (P) then return S(P);
  else
  { Divide the problem into subproblem P1, P2 ... Pk, k≥1.
    Solve and combine the subproblems.
    return ( DAndC(P1), DAndC (P2) ... DAndC(Pk));
  }
}
```



7+1/2 = 4
4 3
3+1/2 = 4.5
Math.floor(4.5)
4 4
mid: = ⌊(i+j)/2⌋;


```
Algorithm MaxMin (i, j, max, min)
{ //a[1:n] is a global array.
  // Parameters i and j are integers
  // 1≤ i ≤ j ≤ n
  // the effect is to set max & min to largest & smallest values in a[i:j]
  if (i=j) then
    max: = min: = a[i];
  else if (i=j-1) then
  { if(a[i] > a[j]) then
    { max:= a[i];
      min: = a[j];
    }
    else
    { max: = a[j];
      min: = a[i];
    }
  }
  else
  { mid: = ⌊(i+j)/2⌋;                    ⌊ ⌋ - floor
    MaxMin (i, mid, max, min);
    MaxMin (mid+1, j, max1, min1);
    if ( max1 > max) then
      max:= max1;
    if (min1 < min) then
      min:= min1;
  }
}
```

[-80, -80, 74, 65, 24, 63, 95, -63, 87, -41]



i=1, j=2, min=-80, max=-80 ✓

i=3, j=3, min=74, max=74 ✓

i=1, j=3, min=-80, max=74 ✓

i=4, j=5, min=24, max=65 ✓

i=1, j=5, min=-80, max=74 ✓

i=6, j=7, min=63, max=95 ✓

i=8, j=8, min=-63, max=-63 ✓

i=6, j=8, min=-63, max=95 ✓

i=9, j=10, min=-41, max=87 ✓

i=6, j=10, min=-63, max=95 ✓

i=1, j=10, min=-80, max=95 ✓

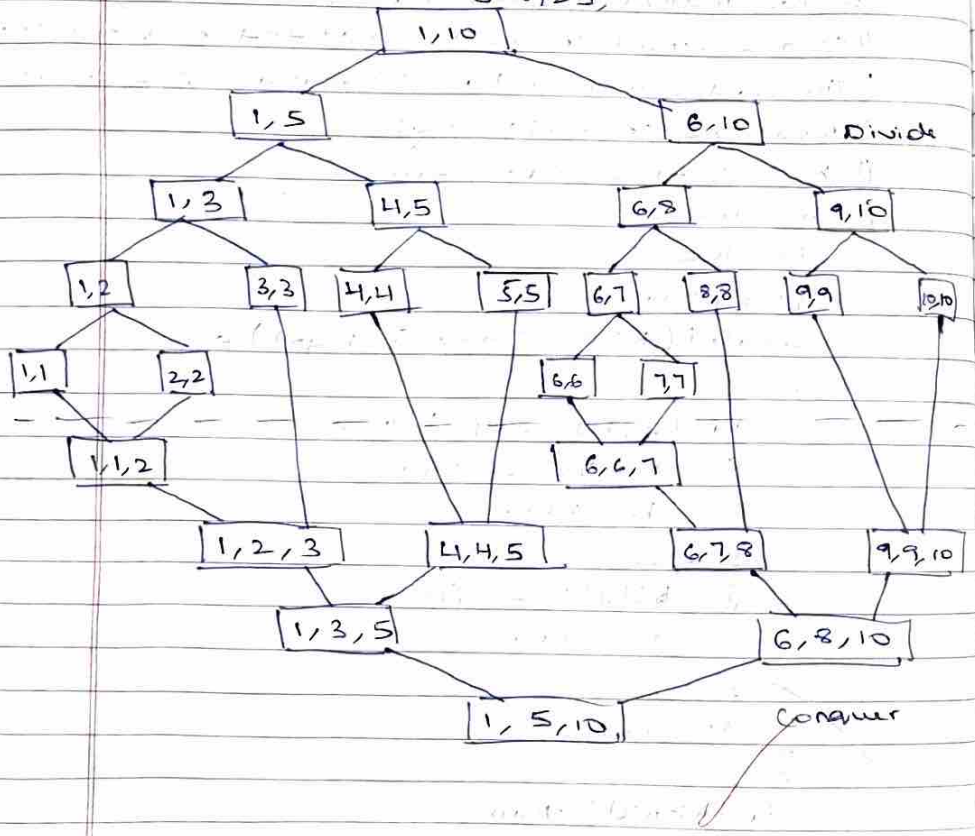
Final Result :- Max=95, Min=-80

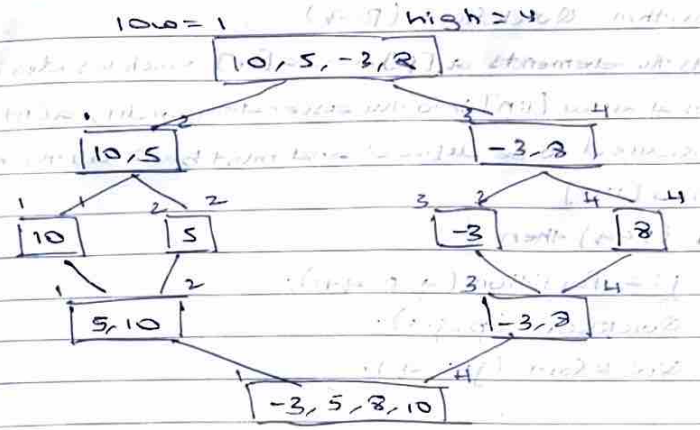
Pass
5/8/26

Merge Sort

```
Algorithm MergeSort (low, high)
// a[low:high] is a global array to be sorted
// small(P) is true if there is one element to sort.
{   if (low<high) then
    {
      mid := ⌊(low+high)/2⌋;
      MergeSort (low, mid);
      MergeSort (mid+1, high);
      Merge (low, mid, high);
    }
}
Algorithm Merge (low, mid, high)
// a[low:high] is a global array containing two
// sorted subsets in a[low:mid] and in a[mid+1:high]
// The goal is to merge these 2 sets into a single set
// resulting in a[low:high]
// b[] is an auxillary array.
{   h := low;
    i := low;
    j := mid+1;
    while ((h ≤ mid) and (j ≤ high)) do
    {
      if (a[h] ≤ a[j]) then
      {   b[i] := a[h];
          h := h+1;
      } else
      {   b[i] := a[j];
          j := j+1;
      }
      i := i+1;
    }
    if (h > mid) then
```

```
for k:=j to high do
{   b[i]:=a[k];
    i:=i+1;
}
else
    for k:=h to mid do
    {   b[i]:=a[k];
        i:=i+1;
    }
for k:=low to high do
    a[k]:=b[k]
}
mid=⌊(low+high)/2⌋;
```





[X, N, N, T, X, F, Ø, X, V, L, E]

XNNTXFØXVLE	XNNTXFØXVLE
XNNTXF|ØXVLE	XNNTXF|ØXVLE
XNN|TXF|ØXVLE	XNN|TXF|ØXVLE
XN|N|TXF|ØXVLE	XN|N|TXF|ØXVLE
X|N|N|TXF|ØXVLE	X|N|N|TXF|ØXVLE
NX|N|TXF|ØXVLE	XN|N|TXF|ØXVLE
NNX|TXF|ØXVLE	XNN|TX|F|ØXVLE
NNX|TX|F|ØXVLE	XNN|T|X|F|ØXVLE
NNX|T|X|F|ØXVLE	XNN|XT|F|ØXVLE
NNX|TX|F|ØXVLE	XNN|XTF|ØXVLE
NNX|FTX|ØXVLE	XXTNNF|ØXVLE
FNNTXX|ØXVLE	XXTNNF|ØXV|LE
FNNTXX|ØXV|LE	XXTNNF|Ø|X|V|LE
FNNTXX|Ø|X|V|LE	XXTNNF|Ø|X|V|LE
FNNTXX|Ø|X|V|LE	XXTNNF|XØ|V|LE
FNNTXX|ØX|V|LE	XXTNNF|XVØ|LE
FNNTXX|ØVX|LE	XXTNNF|XVØ|LE
FNNTXX|ØVX|L|E	XXTNNF|XVØ|LE
FNNTXX|ØVX|EL	XXTNNF|XVØLE
FNNTXX|EL|ØVX	XXTNNF|XVØLE
EFLNNØTVXXX	

QuickSort

```
Algorithm QuickSort (p,q)
// Sorts the elements a[p], ..., a[q] which resides in the
// global array [1:n] into the ascending order, a[n+1] is
// considered to be defined and must be ≥ all the elements
// in a[1:n]
{ if (p<q) then
  { j := Partition (a,p,q+1);
    QuickSort (p,j-1);
    QuickSort (j+1,q);
  }
}
Algorithm Partition (a,m,p)
// Within a[m], a[m+1], ..., a[p-1] the elements are rearrang
// in such a manner that if initially t = a[m] then after completion
// a[q] = t for some q between m and p-1
// a[k] ≤ t for m ≤ k < q and a[k] ≥ t for q < k < p
// q is returned.
{ v := a[m];
  i := m;
  j := p;
  repeat
  { repeat
      i := i+1
    until (a[i] ≥ v);
    repeat
      j := j-1;
    until (a[j] ≤ v);
    if (i < j) then
      Interchange (a,i,j);
  } until (i ≥ j);
  a[m] := a[j];
  a[j] := v;
  return j;
}
```



```
Algorithm Interchange (a,i,j)
{ temp := a[i];
  a[i] := a[j];
  a[j] := temp;
}
```

[10, -3 5 18 2]

[10 -3 5 2 18]

[2 -3 5 10 18]

[2 -3 5] 10 [18]

[-3 2 5] 10 [18]

-3 2 [5] 10 [18]

-3 2 5 10 [18]

-3 2 5 10 18

[452, 602, 991, 396, 894, 763, 335, 375, 853, 910, 676]

[452, 602, 991, 396, 894, 763, 335, 375, 853, 910, 676]

452, 375, 991, 396, 894, 763, 335, 602, 853, 910, 676

452, 375, 335, 396, 894, 763, 991, 602, 853, 910, 676

[396, 375, 335] 452 [894, 763, 991, 602, 853, 910, 676]

[335, 375, 396] 452 [894, 763, 991, 602, 853, 910, 67

[335, 375] 396, 452 [894, 763, 991, 602, 853, 910, 67

335, 375, 396, 452, [894, 763, 991, 602, 853, 910, 676]

335, 375, 396, 452, [894, 763, 676, 602, 853, 910, 991]

335, 375, 396, 452, [853, 763, 676, 602], 894, [910, 991]

335, 375, 396, 452, [602, 763, 676, 853], 894, [910, 991]

335, 375, 396, 452 [602, 763, 676], 853, 894, [910, 991]

335, 375, 396, 452, 602, [763, 676], 853, 894, [910, 991]

335, 375, 396, 452, 602, [676, 763], 853, 894, [910, 991]

335, 375, 396, 452, 602, 676, 763, 853, 894, 910, 991

Descending

[452, 602, 991, 396, 894, 763, 335, 375, 853, 910, 676]

[452, 602, 991, 676, 894, 763, 335, 375, 853, 910, 396]

[452, 602, 991, 676, 894, 763, 910, 375, 853, 335, 396]

[452, 602, 991, 676, 894, 763, 910, 853, 375, 335, 396]

[853, 602, 991, 676, 894, 763, 910], 452, [375, 335, 396]

[853, 910, 991, 676, 894, 763, 602], 452, [375, 335, 396]

[853, 910, 991, ~~894, 676, 763~~, 602], 452, [375, 335, 396]

~~[894, 910, 991, 853, 676, 602], 452, [375, 335, 396]~~

[894, 910, 991], 853, [676, 763, 602], 452, [375, 335, 396]

[991, 910], 894, 853, [676, 763, 602], 452, [375, 335, 396]

991, 910, 894, 853, [763], 676, [602], 452, [375, 335, 396]

991, 910, 894, 853, 763, 676, 602, 452, [375, 335, 396]

991, 910, 894, 853, 763, 676, 602, 452, [~~396~~, 395, ~~335~~]

991, 910, 894, 853, 763, 676, 602, 452, [396], 375, [335]

991, 910, 894, 853, 763, 676, 602, 452, 396, 375, 335

Pass

13/8/24

Strassen's Matrix Multiplication

Matrix Multiplication

- Conventional

$$C(i,j) = \sum_{k=1}^{n} a(i,k) * b(k,j)$$

```
Algorithm MM(a, b, n)
{
    for i := 1 to n do
        for j := 1 to n do
        {   C(i,j) := 0;
            for k := 1 to n do
                c(i,j) = c(i,j) + a(i,k) * b(k,j);
        }
}
```

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$C_{11} = A_{11} * B_{11} + A_{12} * B_{21}$

$C_{12} = A_{11} * B_{12} + A_{12} * B_{22}$

$C_{21} = A_{21} * B_{11} + A_{22} * B_{21}$

$C_{22} = A_{21} * B_{12} + A_{22} * B_{22}$

8 Multiplications

4 Additions.

$$T(n) = 8T\left(\frac{n}{2}\right) + 4n^2$$

4×4 matrix ⇒ 4 2×2 matrices.

8×8 matrix ⇒ 4 4×4 matrices

```
Algorithm MM(A, B, n)
{   if (n <= 2) then
    {C11 = A11 * B11 + A12 * B21
     C12 = A11 * B12 + A12 * B22
     C21 = A21 * B11 + A22 * B21
     C22 = A21 * B12 + A22 * B22
    }
    else
    {
```

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

$O(n^{2.81})$

MM1 $(A_{11}, B_{11}, n/2) + MM(A_{12}, B_{21}, n/2)$
$+ MM(A_{11}, B_{12}, n/2) + MM(A_{12}, B_{22}, n/2)$
$+ MM(A_{21}, B_{11}, n/2) + MM(A_{22}, B_{21}, n/2)$
$+ MM(A_{21}, B_{12}, n/2) + MM(A_{22}, B_{22}, n/2)$

3

3

$P = (A_{11} + A_{22}) * (B_{11} + B_{22})$
$Q = (A_{21} + A_{22}) * B_{11}$
$R = A_{11} * (B_{12} - B_{22})$
$S = A_{22} * (B_{21} - B_{11})$
$T = (A_{11} + A_{12}) * B_{22}$
$U = (A_{21} - A_{11}) * (B_{11} + B_{12})$
$V = (A_{12} - A_{22}) * (B_{21} + B_{22})$

7 multiplications, 18 additions & subtractions.

$C_{11} = P + S - T + V$
$C_{12} = R + T$
$C_{21} = Q + S$
$C_{22} = P + R - Q + V$

① $\begin{bmatrix} 16 & 6 \\ 12 & 11 \end{bmatrix} \begin{bmatrix} 4 & 12 \\ 9 & 6 \end{bmatrix}$

② $\left[\begin{array}{cc|cc} 11 & 8 & 1 & 7 \\ 18 & 16 & 17 & 9 \\ \hline 16 & 12 & 16 & 1 \\ 12 & 19 & 10 & 8 \end{array}\right] \left[\begin{array}{cc|cc} 9 & 16 & 19 & 20 \\ 7 & 14 & 4 & 11 \\ \hline 10 & 11 & 17 & 8 \\ 5 & 16 & 9 & 7 \end{array}\right]$

① $P = (16+11)*(4+6)$
$= 27 * 10 = 270$
$R = 16 * (12 - 6)$
$= 16 * 6 = 96$
$T = (16+6)*6$
$= 22 * 6 = 132$
$V = (6-11)*(9+6)$
$= -5 * 15 = -75$
$Q = (12+11)*4$
$= 23 * 4 = 92$
$S = 11 * (9-4)$
$= 11 * 5 = 55$
$U = (12-16)*(4+12)$
$= -4 * 16 = -64$

$C_{11} = 270 + 55 - 132 + 75 = 118$
$C_{12} = 96 + 132 = 228$
$C_{21} = 55 + 92 = 147$
$C_{22} = 270 + 96 - 92 + -64 = 210$

$\begin{bmatrix} 118 & 228 \\ 147 & 210 \end{bmatrix}$

② $P = \left(\begin{bmatrix} 11 & 8 \\ 18 & 16 \end{bmatrix} + \begin{bmatrix} 6 & 1 \\ 10 & 8 \end{bmatrix}\right) * \left(\begin{bmatrix} 9 & 16 \\ 7 & 14 \end{bmatrix} + \begin{bmatrix} 11 & 13 \\ 9 & 7 \end{bmatrix}\right)$
$= \begin{bmatrix} 17 & 9 \\ 28 & 24 \end{bmatrix} * \begin{bmatrix} 26 & 29 \\ 16 & 21 \end{bmatrix} = \begin{bmatrix} 586 & 682 \\ 1112 & 1316 \end{bmatrix}$

---

① $\begin{bmatrix} 13 & 6 \\ 14 & 2 \end{bmatrix} \begin{bmatrix} 9 & 5 \\ 6 & 11 \end{bmatrix}$

② $\left[\begin{array}{cc|cc} 10 & 3 & 1 & 10 \\ 15 & 9 & 19 & 9 \\ \hline 14 & 11 & 16 & 2 \\ 14 & 18 & 13 & 10 \end{array}\right] \left[\begin{array}{cc|cc} 14 & 8 & 4 & 13 \\ 11 & 11 & 15 & 1 \\ \hline 2 & 9 & 10 & 20 \\ 14 & 16 & 14 & 19 \end{array}\right]$

① $\begin{bmatrix} 13 & 6 \\ 14 & 2 \end{bmatrix} \begin{bmatrix} 9 & 5 \\ 6 & 11 \end{bmatrix} = \begin{bmatrix} 153 & 131 \\ 138 & 92 \end{bmatrix}$

$P = 300$
$Q = 144$
$R = -78$
$S = -6$
$T = 209$
$U = 14$
$V = 68$

$C_{11} = 153$
$C_{12} = 131$
$C_{21} = 138$
$C_{22} = 92$

Pass
19/8/26

② $\left[\begin{array}{cc|cc} 10 & 3 & 1 & 10 \\ 15 & 9 & 19 & 9 \\ \hline 14 & 11 & 16 & 2 \\ 14 & 18 & 13 & 10 \end{array}\right] \left[\begin{array}{cc|cc} 14 & 8 & 4 & 13 \\ 11 & 11 & 15 & 1 \\ \hline 2 & 9 & 10 & 20 \\ 14 & 16 & 14 & 19 \end{array}\right]$

$$P = \left(\begin{bmatrix} 10 & 3 \\ 15 & 9 \end{bmatrix} + \begin{bmatrix} 16 & 2 \\ 13 & 10 \end{bmatrix}\right) * \left(\begin{bmatrix} 14 & 18 \\ 11 & 11 \end{bmatrix} + \begin{bmatrix} 10 & 20 \\ 14 & 19 \end{bmatrix}\right)$$

$$= \begin{bmatrix} 26 & 5 \\ 28 & 19 \end{bmatrix} * \begin{bmatrix} 24 & 28 \\ 25 & 30 \end{bmatrix} = \begin{bmatrix} 624+125 & 728+150 \\ 672+475 & 784+570 \end{bmatrix}$$

$$= \begin{bmatrix} 749 & 878 \\ 1147 & 1354 \end{bmatrix}$$

$$Q = \left(\begin{bmatrix} 14 & 11 \\ 14 & 18 \end{bmatrix} + \begin{bmatrix} 16 & 2 \\ 13 & 10 \end{bmatrix}\right) * \begin{bmatrix} 14 & 8 \\ 11 & 11 \end{bmatrix}$$

$$= \begin{bmatrix} 30 & 13 \\ 27 & 28 \end{bmatrix} * \begin{bmatrix} 14 & 8 \\ 11 & 11 \end{bmatrix} = \begin{bmatrix} 420+143 & 240+143 \\ 378+308 & 216+308 \end{bmatrix}$$

$$= \begin{bmatrix} 563 & 383 \\ 686 & 524 \end{bmatrix}$$

$$R = \begin{bmatrix} 10 & 3 \\ 15 & 9 \end{bmatrix} * \left(\begin{bmatrix} 4 & 13 \\ 15 & 1 \end{bmatrix} - \begin{bmatrix} 10 & 20 \\ 14 & 19 \end{bmatrix}\right)$$

$$= \begin{bmatrix} 10 & 3 \\ 15 & 9 \end{bmatrix} * \begin{bmatrix} -6 & -7 \\ 1 & -18 \end{bmatrix} = \begin{bmatrix} -60+3 & -70-54 \\ -90+9 & -105-162 \end{bmatrix}$$

$$= \begin{bmatrix} -57 & -124 \\ -81 & -267 \end{bmatrix}$$

$$S = \begin{bmatrix} 16 & 2 \\ 13 & 10 \end{bmatrix} * \left(\begin{bmatrix} 2 & 9 \\ 14 & 16 \end{bmatrix} - \begin{bmatrix} 14 & 8 \\ 11 & 11 \end{bmatrix}\right)$$

$$= \begin{bmatrix} 16 & 2 \\ 13 & 10 \end{bmatrix} * \begin{bmatrix} -12 & 1 \\ 3 & 5 \end{bmatrix} = \begin{bmatrix} -192+6 & 16+10 \\ -156+30 & 13+50 \end{bmatrix}$$

$$= \begin{bmatrix} -186 & 26 \\ -126 & -37 \end{bmatrix} \quad (26), \ (63)$$

$$T = \left(\begin{bmatrix} 10 & 3 \\ 15 & 9 \end{bmatrix} + \begin{bmatrix} 1 & 10 \\ 19 & 9 \end{bmatrix}\right) * \begin{bmatrix} 10 & 20 \\ 14 & 19 \end{bmatrix}$$

$$= \begin{bmatrix} 11 & 13 \\ 34 & 18 \end{bmatrix} * \begin{bmatrix} 10 & 20 \\ 14 & 19 \end{bmatrix} = \begin{bmatrix} 110+182 & 220+247 \\ 340+252 & 680+342 \end{bmatrix}$$

$$= \begin{bmatrix} 292 & 467 \\ 592 & 1022 \end{bmatrix}$$

$$U = \left(\begin{bmatrix} 14 & 11 \\ 14 & 18 \end{bmatrix} - \begin{bmatrix} 10 & 3 \\ 15 & 9 \end{bmatrix}\right) * \left(\begin{bmatrix} 14 & 8 \\ 11 & 11 \end{bmatrix} + \begin{bmatrix} 4 & 13 \\ 15 & 1 \end{bmatrix}\right)$$

$$= \begin{bmatrix} 4 & 8 \\ -1 & 9 \end{bmatrix} * \begin{bmatrix} 18 & 21 \\ 26 & 12 \end{bmatrix} = \begin{bmatrix} 72+208 & 84+96 \\ -18+234 & -21+108 \end{bmatrix}$$

$$= \begin{bmatrix} 280 & 180 \\ 216 & 87 \end{bmatrix}$$

$$V = \left(\begin{bmatrix} 1 & 10 \\ 19 & 9 \end{bmatrix} - \begin{bmatrix} 16 & 2 \\ 13 & 10 \end{bmatrix}\right) * \left(\begin{bmatrix} 2 & 9 \\ 14 & 16 \end{bmatrix} + \begin{bmatrix} 10 & 20 \\ 14 & 19 \end{bmatrix}\right)$$

$$= \begin{bmatrix} -15 & -1 \\ 6 & -1 \end{bmatrix} * \begin{bmatrix} 12 & 29 \\ 28 & 35 \end{bmatrix} = \begin{bmatrix} -180+224 & -435+280 \\ 72-28 & 174-25 \end{bmatrix}$$

$$= \begin{bmatrix} 44 & -235 \\ 44 & 149 \end{bmatrix} \quad (-155), \ (139)$$

$$C_{11} = \begin{bmatrix} 749 & 878 \\ 1147 & 1354 \end{bmatrix} + \begin{bmatrix} -186 & 26 \\ -126 & 63 \end{bmatrix} - \begin{bmatrix} 292 & 467 \\ 592 & 1022 \end{bmatrix}$$

$$+ \begin{bmatrix} 44 & -155 \\ 44 & 139 \end{bmatrix} = \begin{bmatrix} 315 & 282 \\ 473 & 534 \end{bmatrix} \quad (182), \ (444)$$

$$C_{12} = \begin{bmatrix} -57 & -124 \\ -81 & -267 \end{bmatrix} + \begin{bmatrix} 292 & 467 \\ 592 & 1022 \end{bmatrix} = \begin{bmatrix} 235 & 343 \\ 511 & 755 \end{bmatrix}$$

$$C_{21} = \begin{bmatrix} 563 & 383 \\ 686 & 524 \end{bmatrix} + \begin{bmatrix} -186 & 26 \\ -126 & 63 \end{bmatrix} = \begin{bmatrix} 377 & 409 \\ 560 & 587 \end{bmatrix} \quad (389), \ (487)$$

$$C_{22} = \begin{bmatrix} 749 & 878 \\ 1147 & 1354 \end{bmatrix} + \begin{bmatrix} -57 & -124 \\ -81 & -267 \end{bmatrix} - \begin{bmatrix} 563 & 383 \\ 686 & 524 \end{bmatrix}$$

$$+ \begin{bmatrix} 280 & 180 \\ 216 & 87 \end{bmatrix} = \begin{bmatrix} 409 & 87 \\ 596 & 650 \end{bmatrix} \quad (551)$$

$$= \begin{bmatrix} 315 & 282 \\ 473 & 534 \end{bmatrix} \quad (182) \quad \begin{bmatrix} 235 & 343 \\ 511 & 755 \end{bmatrix}$$

$$\begin{bmatrix} 377 & 409 \\ 560 & 587 \end{bmatrix} \quad (389) \quad \begin{bmatrix} 409 & 551 \\ 596 & 650 \end{bmatrix}$$

Page 02/09/20

Greedy Method

GM is used to solve optimization problem.

The problem which requires either ~~min~~ min^m or max^m result. The problem has 'n' inputs and require us to obtain a subset that satisfies some constraint.

Any subset that satisfies these constraints is called a feasible soln.

We need to find a feasible soln that either maximises or minimizes a given objective fn. A feasible soln that does this is called an optimal soln.

The greedy method suggests that one can devise an algorithm that works in stages considering one input at a time. At each stage, decision is made regarding whether a particular input is an optimal solution.

The selection procedure is based on some optimization measure. This measure may be the objective fn.

This version of the greedy technique is called the subset paradigm.

~~Control~~

GM Control Abstraction for the subset paradigm.

```
Algorithm Greedy (a,n)
// a[1:n] contains n inputs
{   solution := ϕ
    for i:=1 to n do:
    { x := Select(a);
      if feasible(solution,x) then
          solution := Union(solution, x);
    }
    return solution;
}
```


The fn Select, selects an I/P from array a & removes it. The selected I/P is assigned to x. Feasible is a boolean valued fn that determines whether x can be included into the soln vector. The fn union combines x with the solution & updates objective fn.

Knapsack

We are given n objects and a knapsack. Object i has weight w_i & knapsack has capacity m. If a fraction x_i, $0 \le x_i \le 1$, of object i is placed into the knapsack then a profit of $p_i x_i$ is earned. The objective is to obtain a filling of the knapsack that maximises the total profit earned. Since the knapsack capacity is m, we require total weight of all chosen objects to be at the most m. Formally the problem can be stated as

maximize $\sum_{1 \le i \le n} p_i x_i$ — (1)

subjected to $\sum_{1 \le i \le n} w_i x_i \le m$ — (2)

and $0 \le x_i \le 1$, $1 \le i \le n$ — (3)

A feasible soln is any set $(x_1, \ldots, x_n)$ satisfying (2) & (3).

An optimal soln is a feasible soln for which (1) is maximized.

Fractions → $\frac{1}{2}, \frac{1}{3}, \frac{1}{4}$

for n elements = $\frac{1}{n+1}$ upto

$m = 20, n = 3 \quad (P_1, P_2, P_3) = (25, 24, 15)$

$(w_1, w_2, w_3) = (18, 15, 10)$

Soln vector

Strategy (x_1, x_2, x_3)	$\sum w_i x_i$	$\sum p_i x_i$
Fraction $(\frac{1}{2}, \frac{1}{3}, \frac{1}{4})$	$18\times\frac{1}{2} + 15\times\frac{1}{3} + 10\times\frac{1}{4}$ $= 16.5$	$25\times\frac{1}{2} + 24\times\frac{1}{3} + 15\times\frac{1}{4}$ $= 24.25$
Max Profit $(1, 2/15, 0)$	$18\times1 + 15\times\frac{2}{15} + 0$ $= 20$	$= 28.2$
least wt $(0, 2/3, 1)$	20	31
Ratio P_i/w_i $(0, 1, 1/2)$	20	31.5

where $\frac{P_i}{w_i} \geq \frac{P_{i+1}}{w_{i+1}}$

$\frac{25}{18}, \frac{24}{15}, \frac{15}{10}$

$= 1.3 = 1.6 = 1.5$

③ ① ②

Capacity = 648

weight	value
20	599
85	994
82	389
67	852
92	719
86	819
43	177
44	216
36	200
68	648
47	911
670	

Strategy $(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11})$ $\sum w_i x_i$ $\sum p_i x_i$

Fraction $(\frac{1}{2}, \frac{1}{3}, \ldots\ldots\ldots, \frac{1}{12})$ 123.78 1336

$x_7 =$ 123.81 1336

Max Profit $(1, 1, 1, 1, 1, 1, 21/43, 1, 1, 1, 1)$ 648 6483.4

$x_5 = \frac{25}{35}$

least wei ght $(1, 1, 1, 1, \frac{20}{46}, 1, 1, 1, 1, 1, 1)$ 648 6352.0

$x_7 =$

Ratio P_i/w_i $(1, 1, 1, 1, 1, 1, 21/43, 1, 1, 1, 1)$ 648 6433.4

① 29.95 599 20 ⑦ 11.69 994 85 ⑩ 4.74 389 82 ⑤ 12.71 852 67 ⑦ 7.81 719 92 ⑥ 9.52 819 86 ⑪ 4.71 177 43 ⑨ 4.90 216 44

③ 5.55 200 36 ⑧ 9.52 648 68 ⑫ 19.38 911 47

$\frac{1}{2}\times 20 + \frac{1}{3}\times 85 + \frac{1}{4}\times 82 + \frac{1}{5}\times 64 + \frac{1}{6}\times 92 + \frac{1}{7}\times 86 + \frac{1}{8}\times 43$

$+ \frac{1}{9}\times 44 + \frac{1}{10}\times 36 + \frac{1}{11}\times 68 + \frac{1}{12}\times 47 =$

$10 + 28.33 + 20.5 + 13.4 + 15.33 + 12.28 + 5.37 + 4.88$

$+ 3.6 + 6.18 + 3.91 = 123.78$

$\frac{1}{2}\times 599 + \frac{1}{3}\times 994 + \frac{1}{4}\times 389 + \frac{1}{5}\times 852 + \frac{1}{6}\times 719 + \frac{1}{7}\times 819 +$

$\frac{1}{8}\times 177 + \frac{1}{9}\times 216 + \frac{1}{10}\times 200 + \frac{1}{11}\times 648 + \frac{1}{12}\times 911 =$

$299.5 + 331.33 + 97.25 + 170.4 + 119.83 + 117 + 22.125 +$

$24 + 20 + 58.90 + 75.91 = 1336.24$

Pass

02/09/20

```
Algorithm Greedy_knap (m,n)
// p[1...n] and w[1:n] contain the profits & weights
// respectively of the n objects ordered such that
// p[i]/w[i] ≥ p[i+1]/w[i+1]
//m is knapsack size & x[1:n] is a solution vector
{
  for i:=1 to n do
  { x[i]:=0.0;
    u:=m;
  }
  for i:=1 to n do
  {
    if (w[i]>u) then break;
    x[i]:=1.0;
    u:=u-w[i];
  }
  if (i≤n) then
    x[i]:=u/w[i];
}
```

Page 20
24/09/20

Prims MST ($O(n^2)$) 2/9/26

```
Algorithm Prim (E, cost, n, t)
// where E is the set of edges in G. cost[1:n,1:n]
// is the cost adjacency matrix of n vertex graph
// such that cost[i,j] is either positive real number or
// ∞ if no edge (i,j) exists. A MST is computed
// and stored as a set of edges in the array t[1:n-1,
// 1:2]. The edge (t[i,1], t[i,2]) is an edge in the
// minimum cost spanning tree. The final cost is returned
{
    let (k,l) be an edge of minimum cost in E;        ⇒O(|E|)
    mincost:=cost[k,l];                               ⇒O(1)
    t[1,1]:=k;
    t[1,2]:=l;
    for i:=1 to n do                                  ⇒Θ(n)
      if cost[i,l] < cost[i,k] then
        near[i]:=l;
      else
        near[i]:=k;
    near[k]:=0;
    near[l]:=0;
    for i:=2 to n-1 do                                ⇒O(n²)
    {
      let j be an index s.t near[j]≠0 & cost[j,near[j]]
      is minimum;                                     ⇒O(n)
      t[i,1]:=j;
      t[i,2]:=near[j];
      mincost:=mincost+cost[j,near[j]];
      near[j]:=0;
      for k:=1 to n do
        if((near[k]≠0) and (cost[k,near[k]] >
              cost[k,j])) then
          near[k]:=j;
    } // for i:=2
    return mincost;
}
```

	1	2	3	4	5	6	7	8	9	10
1	INF	22	48	50	68	75	31	13	88	99
2	22	INF	CC	81	51	93	69	48	87	78
3	48	86	INF	95	24	86	99	96	53	61
4	50	81	95	INF	INF 59 57 ~~59 57 54~~			54	58	83
5	68	51	24	INF	INF	~~INF 57 63 INF~~ 57 63 INF 45				48
6	75	93	86	59	57	INF	INF	47	80	87
7	31	69	99	57	63	INF	INF	23	45	23
8	13	48	96	54	INF	47	23	INF	INF	53
9	88	87	53	58	45	80	45	INF	INF	INF
10	99	78	61	83	48	87	23	53	INF	INF

① 1 8

mincost = 13

near[k] 0 1 1 1 1 8 8 0 1 8

cost[k, near[k]] – 22 48 50 68 47 23 – 88 53 j = 7

② 1 8

2 1

mincost = 35

near[k] 0 0 1 1 2 8 8 0 2 8

cost[k, near[k]] – – 48 50 51 47 23 – 87 53 j = 7

③ 1 8

2 1

7 8

mincost = 58

near[k] 0 0 1 1 2 8 0 0 7 7

cost[k, near[k]] – – 48 50 51 47 – – 45 23 j = 10

③ 1 8

2 1

7 8

10 7

mincost = 81

near[k] 0 0 1 10 8 0 0 7 0

cost[k, near[k]] – – 48 50 47 – – 45 – j = 9

④ 1 8

2 1

7 8

10 7

9 7

mincost = 126

near[k] 0 0 1 1 9 8 0 0 0 0

cost[k, near[k]] – – 48 50 45 47 – – – – j = 5

⑤ 1 8

2 1

7 8

10 7

9 7

5 9

mincost = 171

near[k] 0 0 5 1 0 8 0 0 0 0

cost[k, near[k]] – – 24 50 – 47 – – – – j = 3

⑥ 1 8

2 1

7 8

10 7

9 7

5 9

8 5

mincost = 195

near[k] 0 0 0 1 0 8 0 0 0 0

cost[k, near[k]] – – – 50 – 47 – – – – j = 6

(4) (7) 1 8
2 1
7 8
10 7
9 7
5 9
3 5
6 8

mincost = 242

near[k] 0 0 0 1 0 0 0 0 0 0

cost[k, near[k]] — — — 50 — — — — — —

i = 4

(8) 1 8
2 1
7 8
10 7
9 7
5 9
3 5
6 8
4 1

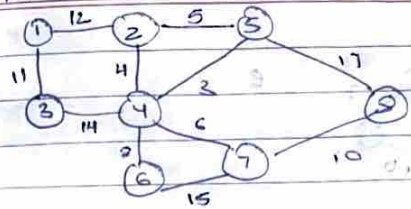
mincost = 292

near[k] 0 0 0 0 0 0 0 0 0 0

cost[k, near[k]] — — — — — — — — — —

Pas
02/09/26

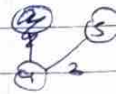
Prim's MST



(1)

t	1	2
1	4	5
2	2	4

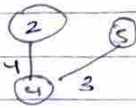
mincost = 2



(2)

t	1	2
1	4	5
2	2	4
3	7	4

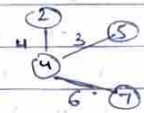
mincost = 7



(3)

t	1	2
1	4	5
2	2	4
3	7	4

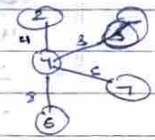
mincost = 13



(4)

t	1	2
1	4	5
2	2	4
3	7	4
4	6	4

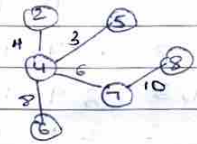
mincost = 21



(5)

t	1	2
1	4	5
2	2	4
3	7	4
4	6	4
5	8	7

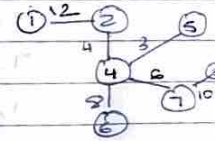
mincost = 31



(6)

t	1	2
1	4	5
2	2	4
3	7	4
4	6	4
5	8	7
6	1	2

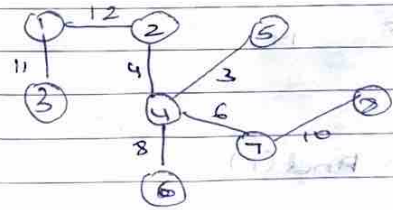
mincost = 43



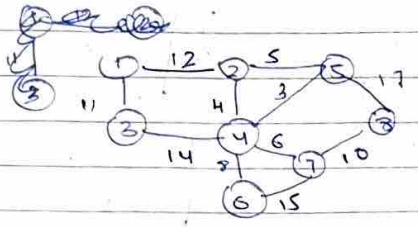
(7)

t	1	2
1	4	5
2	2	4
3	7	4
4	6	4
5	8	7
6	1	2
7	3	1

mincost = 54



Kruskal's MST 23/9/26



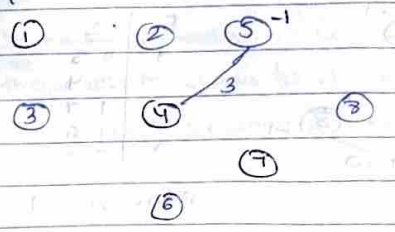
E

Edges	Cost	Parent :- [1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
4,5	3	UV	-1	-1	-1	-1	-1	-1	-1
2,4	4	4,5	-1	-1	-1	5	-1	-1	-1
2,5	5	2,4	-1	4	-1	5	-1	-1	-1
4,7	6	4,7	-1	4	-1	5	-1	-1	-1
4,6	8	4,6	-1	4	-1	6	-1	-1	-1
7,8	10	7,8	-1	4	-1	6	-1	-1	8
1,3	11	1,3	3	4	-1	6	-1	-1	8
1,2	12	1,2	2	4	-1	6	-1	-1	8
3,4	14								
6,7	15								
5,8	17								

Algorithm Find(i)
{
while parent[i] ≥ 0 do
i := parent[i];
return i;
}

Algorithm Union(i,j)
{
parent[i] := j;
}

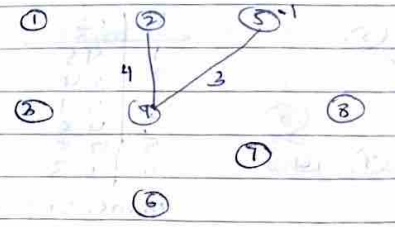
Step 1



T	1	2
1	4	5

Mincost = 3

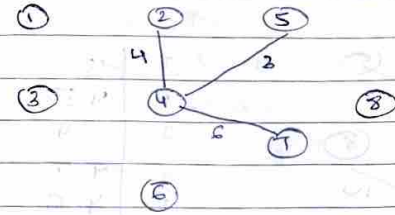
Step 2



t	1	2
1	4	5
2	2	4

Mincost = 7

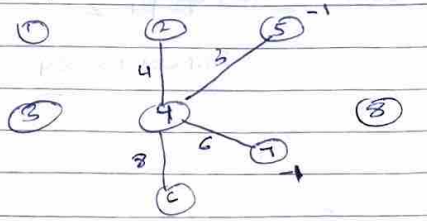
Step 3



t	1	2
1	4	5
2	2	4
3	4	7

Mincost = 13

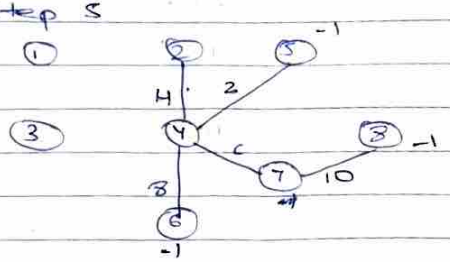
Step 4



t	1	2
1	4	5
2	2	4
3	4	7
4	4	6

mincost = 21

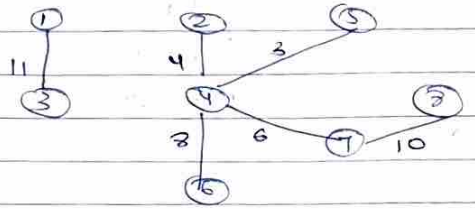
Step 5



t	1	2
1	4	5
2	2	4
3	4	7
4	4	6
5	7	8

Mincost = 31

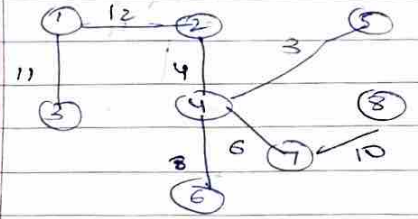
Step 6



t	1	2
1	4	5
2	2	4
3	4	7
4	4	6
5	7	8
6	1	3

Mincost = 42

Step 7



t	1	2
1	4	5
2	2	4
3	4	7
4	4	6
5	7	8
6	1	3
7	1	2

Mincost = 54

=

for 23/9/20


```
Algorithm Kruskals (E, cost, n, t)
// E is the set of edges in G; G has n vertices. cost[u,v]
// is the cost of edges (u,v). t is the set of edges in the
// minimum cost spanning tree. The final mincost is
// returned.
{
   Construct a heap out of edges using Heapify.
   for i:= 1 to n do
      parent[i]:= -1;
   // Each vertex is in a different set
    i:= 0; mincost:= 0.0;
   while ((i<n-1) and (heap not empty)) do
   {
      delete a minimum cost edge (u,v) from the heap
      and reheapify using Adjust;
      j:= Find(u);
      k:= Find(v);
      if (j ≠ k) then
      { i:= i+1;
        t[i,1]:= u;
        t[i,2]:= v;
        mincost:= mincost + cost[u,v];
        Union(j,k);
      }
   }
   if (i ≠ n-1) then write ("No spanning tree");
   else return mincost;
}
```

```
Algorithm Heapify (a,n)
{  for i := ⌊n/2⌋ to 1 step -1 do
       Adjust (a,i,n);
}
Algorithm Adjust (a,i,n)
{  j := 2i;
   item := a[i];
   while (j≤n) do
   {  if ((j<n) and (a[j]>a[j+1])) then
          j := j+1
      if (item ≤ a[j]) then break;
      a[⌊j/2⌋] := a[j];
      j := 2j;
   }
   a[⌊j/2⌋] := item;
}
Algorithm DelMin (A,n,x)
{  if (n=0) then
   {  write ("heap empty")
      return false;
   }
   x := a[1];
   a[1] := a[n];
   Adjust (a,1,n-1);
   return true;
}
```

Pau 23/9/20

Dijkstra's Shortest Path 30/9/20

```
Algorithm ShortestPath (v,cost,dist,n)
// dist[j], 1≤j≤n is set to the length of the shortest
// path from vertex v to vertex j in a digraph G with n
// vertices. dist[v] is set to 0. G is represented by its cost
// adjacency matrix, cost[1:n,1:n].
{  for i := 1 to n do
   {  S[i] := false
      dist[i] := cost[v,i];
   }
   S[v] := true;
   dist[v] := 0.0;
   for num := 2 to n-1 do
   {  choose u from among those vertices not in S such that
      dist[u] is minimum.
      S[u] := true;
      for (each w adjacent to u with S[w] = false) do
         if dist[w] > dist[u] + cost[u,w]
            dist[w] := dist[u] + cost[u,w];
   }
}
```

	1	2	3	4	5	6	7	8
1	0	49	56	∞	∞	71	88	∞
2	∞	0	∞	∞	39	∞	87	∞
3	∞	∞	0	58	69	∞	∞	94
4	10	∞	82	0	32	22	63	∞
5	∞	94	∞	84	0	59	22	35
6	∞	45	∞	∞	10	0	32	72
7	∞	∞	∞	∞	∞	16	0	64
8	70	∞	38	39	∞	85	95	0

Step 1 v = 1 ✓

S[1] = true S[2] = false S[3] = false S[4] = false
S[5] = false S[6] = false S[7] = false S[8] = false
dist[1] = 0 dist[2] = 49 dist[3] = 56 dist[4] = ∞
dist[5] = ∞ dist[6] = 71 dist[7] = 88 dist[8] = ∞

Step 2 v = 2 ✓

S[1] = true S[2] = true S[3] = false S[4] = false
S[5] = false S[6] = false S[7] = false S[8] = false
dist[1] = 0 dist[2] = 49 dist[3] = 56 dist[4] = ∞
dist[5] = 88 dist[6] = 71 dist[7] = 88 dist[8] = ∞

Step 3 v = 3 ✓

S[1] = true S[2] = true S[3] = true S[4] = false
S[5] = false S[6] = false S[7] = false S[8] = false
dist[1] = 0 dist[2] = 49 dist[3] = 56 dist[4] = 114
dist[5] = 88 dist[6] = 71 dist[7] = 88 dist[8] = 150

Step 4 v = 6 ✓

S[1] = true S[2] = true S[3] = true S[4] = false
S[5] = false S[6] = true S[7] = false S[8] = false
dist[1] = 0 dist[2] = 49 dist[3] = 56 dist[4] = 114
dist[5] = 81 dist[6] = 71 dist[7] = 88 dist[8] = 143

Step 5 v = 5 ✓

S[1] = true S[2] = true S[3] = true S[4] = false
S[5] = true S[6] = true S[7] = false S[8] = false
dist[1] = 0 dist[2] = 49 dist[3] = 56 dist[4] = 114
dist[5] = 81 dist[6] = 71 dist[7] = 88 dist[8] = 116

Step 6 v = 7 ✓

S[1] = true S[2] = true S[3] = true S[4] = false
S[5] = true S[6] = true S[7] = true S[8] = false
dist[1] = 0 dist[2] = 49 dist[3] = 56 dist[4] = 114
dist[5] = 81 dist[6] = 71 dist[7] = 88 dist[8] = 116

Step 7 v = 4 ✓

S[1] = true S[2] = true S[3] = true S[4] = true
S[5] = true S[6] = true S[7] = true S[8] = false
dist[1] = 0 dist[2] = 49 dist[3] = 56 dist[4] = 114
dist[5] = 81 dist[6] = 71 dist[7] = 88 dist[8] = 116

Step 8 v = 8 ✓

S[1] = true S[2] = true S[3] = true S[4] = true
S[5] = true S[6] = true S[7] = true S[8] = true
dist[1] = 0 dist[2] = 49 dist[3] = 56 dist[4] = 114
dist[5] = 81 dist[6] = 71 dist[7] = 88 dist[8] = 116

src	dist	length	path
1	2	49	1→2
1	3	56	1→3
1	4	114	1→3→4
1	5	81	1→6→5
1	6	71	1→6
1	7	88	1→7
1	8	116	1→6→5→8

1→6→5→8

	1	2	3	4	5	6	7	8
1	0	13	INF	INF	INF	INF	34	INF
2	10	0	22	INF	10	INF	34	20
3	INF	17	0	20	26	17	22	12
4	INF	16	10	0	INF	18	11	INF
5	35	19	33	25	0	INF	INF	13
6	35	26	20	25	26	0	INF	INF
7	INF	INF	29	17	32	31	0	INF
8	INF	35	INF	16	22	INF	31	0

Step 1 v = 1

S[1] = true S[2] = false S[3] = false S[4] = false
S[5] = false S[6] = false S[7] = false S[8] = false
dist[1] = 0 dist[2] = 13 dist[3] = ∞ dist[4] = ∞
dist[5] = ∞ dist[6] = ∞ dist[7] = 34 dist[8] = ∞

Step 2 v = 2

S[1] = true S[2] = true S[3] = false S[4] = false
S[5] = false S[6] = false S[7] = false S[8] = false
dist[1] = 0 dist[2] = 13 dist[3] = 35 dist[4] = ∞
dist[5] = 23 dist[6] = ∞ dist[7] = 34 dist[8] = 33

Step 3 v = 5

S[1] = true S[2] = true S[3] = false S[4] = false
S[5] = true S[6] = false S[7] = false S[8] = false
dist[1] = 0 dist[2] = 13 dist[3] = 35 dist[4] = 48
dist[5] = 23 dist[6] = ∞ dist[7] = 34 dist[8] = 33

Step 4 v = 8

S[1] = true S[2] = true S[3] = false S[4] = false
S[5] = true S[6] = false S[7] = false S[8] = true
dist[1] = 0 dist[2] = 13 dist[3] = 35 dist[4] = 48
dist[5] = 23 dist[6] = 55 dist[7] = 34 dist[8] = 33

Sat 30/9/20

Steps v = 7

S[1] = true S[2] = true S[3] = false S[4] = false
S[5] = true S[6] = false S[7] = true S[8] = true
dist[1] = 0 dist[2] = 13 dist[3] = 35 dist[4] = 48
dist[5] = 23 dist[6] = 55 dist[7] = 34 dist[8] = 33

v = 3

S[1] = true S[2] = true S[3] = true S[4] = false
S[5] = true S[6] = false S[7] = true S[8] = true
dist[1] = 0 dist[2] = 13 dist[3] = 35 dist[4] = 48
dist[5] = 23 dist[6] = ~~55~~ 52 dist[7] = 34 dist[8] = 33

v = 4

S[1] = true S[2] = true S[3] = true S[4] = true
S[5] = true S[6] = false S[7] = true S[8] = true
dist[1] = 0 dist[2] = 13 dist[3] = 35 dist[4] = 48
dist[5] = 23 dist[6] = 52 dist[7] = 34 dist[8] = 33

v = 6

S[1] = true S[2] = true S[3] = true S[4] = true
S[5] = true S[6] = true S[7] = true S[8] = true
dist[1] = 0 dist[2] = 13 dist[3] = 35 dist[4] = 48
dist[5] = 23 dist[6] = 52 dist[7] = 34 dist[8] = 33

Src	dest	length	Path
1	2	13	1 → 2
1	3	35	1 → 2 → 3
1	4	48	1 → 2 → 5 → 4
1	5	23	1 → 2 → 5
1	6	52	1 → 2 → 3 → 6
1	7	34	1 → 7
1	8	33	1 → 2 → 8

1 → 2 → ..

Multistage Graph

```
Algorithm FGraph (G, k, n, p)
// The input is a k-stage G = (V, E) with index in
// order of stages j. E is a set of edges and
// c[i,j] is cost of <i,j>. p[1:k] is a minimum
// cost path.
{   cost[n] := 0;
    for j := n-1 to 1 step -1 do
    { let r be a vertex s.t <j,r> is an edge of G &
      c[j,r] + cost[r] is minimum.
      cost[j] := c[j,r] + cost[r];
      d[j] := r;
    }
    // Find a minimum cost path
    p[1] := 1; p[k] := n;
    for j := 2 to k-1 do
        p[j] := d[p[j-1]];
}
```

```
Algorithm BGraph (G, k, n, p)
// The input is a k-stage G = (V, E) with index in
// order of stages j. E is a set of edges and
// c[i,j] is cost of <i,j>. p[1:k] is a minimum
// cost path
{   bcost[1] := 0;
    for j := 2 to n do
    { let r be a vertex s.t <r,j> is an edge of G &
      c[r,j] + bcost[r] is minimum;
      cost[j] := c[r,j] + bcost[r];
      d[j] := r;
    }
    // find a minimum cost path
    p[1] := 1; p[k] := n;
    for j := k-1 to 2 step -1 do
        p[j] := d[p[j+1]];
}
```


1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	11	8	1	12											
2					10	7	8								
3					9		12								
4					15	5	6								
5					7	1									
6								9	9						
7								11	4	12					
8							13			8					
9											15	14	4		
10											15	2	8		
11											13	3	8		
12												9			
13															4
14															12
15															12
16															

cost[16] = 0

cost[15] = min(12) = 12

d[15] = 16

cost[14] = min(12) = 12

d[14] = 16

cost[13] = min(4) = 4

d[13] = 16

cost[12] = min(21) = 21

d[12] = 14

cost[11] = min(13+4, 3+12, 8+12) = 15

d[11] = 14

cost[10] = min(19+14, 20) = 14

d[10] = 14

cost[9] = min(19+24, 16) = 16

d[9] = 15

cost[8] = min(29, 29) = 29
d[8] = 9
cost[7] = min(25, 19, 34) = 19
d[7] = 11
cost[6] = min(23, 24) = 23
d[6] = 10
cost[5] = min(30, 20) = 20
d[5] = 7
cost[4] = min(38, 24, 35) = 24
d[4] = 7
cost[3] = min(32, 41) = 32
d[3] = 6
cost[2] = min(33, 26, 37) = 26
d[2] = 7
cost[1] = min(37, 40, 25, 33) = 25
d[1] = 4

Mincost = 25
Path :- 1 → 4 → 7 → 11 → 14 → 16

bcost[1] = 0
bcost[2] = min(11) = 11 d[2] = 1
bcost[3] = min(8) = 8 d[3] = 1
bcost[4] = min(1) = 1 d[4] = 1
bcost[5] = min(13) = 13 d[5] = 1
bcost[6] = min(21, 17, 16, 20) = 16 d[6] = 4
bcost[7] = min(18, 6, 14) = 6 d[7] = 4
bcost[8] = min(19, 20, 7) = 7 d[8] = 4
bcost[9] = min(20) = 20 d[9] = 8
bcost[10] = min(25, 17) = 17 d[10] = 7
bcost[11] = min(25, 19) = 19 d[11] = 7
bcost[12] = min(19, 15) = 15 d[12] = 8
bcost[13] = min(35, 32, 23) = 23 d[13] = 11
bcost[14] = min(34, 19, 13, 24) = 13 d[14] = 11
bcost[15] = min(24, 28, 18) = 18 d[15] = 11
bcost[16] = min(27, 25, 30) = 25 d[16] = 14

mincost = 23
Path :- 16 → 14 → 11 → 7 → 4 → 1

Page 21/10/26

All-pairs SP (Floyd-Warshall)

$A(i,j) = \min\{\min_{1 \le k \le n}\{A^{k-1}(i,k) + A^{k-1}(k,j)\}, cost(i,j)\}$

$A^0(i,j) = cost(i,j) \quad 1 \le i \le n, \ 1 \le j \le n$

$A^k(i,j) = \min\{A^{k-1}(i,j), A^{k-1}(j,k) + A^{k-1}(k,j)\}$

```
Algorithm Allpairs SP (cost, A, n)
// cost[1:n, 1:n] is the adjacency matrix of a graph with n vertices
// A[i,j] is the cost of SP from vertex i to vertex j; cost[i,j]=co
// 1 ≤ i ≤ n;
{ for i:=1 to n do
    for j:=1 to n do
        A(i,j):=cost(i,j);
  for k:=1 to n do
    for i:=1 to n do
      for j:=1 to n do
        if A(i,j) > A(i,k) + A(k,j) then
            A(i,j):= A(i,k) + A(k,j);
}
```

$p^0(i,j) = i \quad i \ne j$
$= - \quad i = j$

$p^k(i,j) = p^{k-1}(k,j)$ if $A^k(i,j) \ne A^{k-1}(i,j)$
$= p^{k-1}(i,j)$ otherwise.

A^0	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	6	0

p^0	1	2	3	4	5
1	-	1	1	1	1
2	2	-	2	2	2
3	3	3	-	3	3
4	4	4	4	-	4
5	5	5	5	5	-

A^1	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

p^1	1	2	3	4	5
1	-	1	1	1	1
2	2	-	2	2	2
3	3	3	-	3	3
4	4	1	4	-	1
5	5	5	5	5	-

Shortest Path
2
1 3
4
1 5
2 1
2 3
2 4
2 5
3 1
3 2
3 4
3 5
4 1
4 2
4 3
4 5
5 1
5 2

A^2

	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

P^2

	1	2	3	4	5
1	1	2	3	2	1
2	2	–	2	2	2
3	3	3	–	2	2
4	4	1	4	–	1
5	5	5	5	5	–

A^3

	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

P^3

	1	2	3	4	5
1	1	1	1	2	1
2	2	–	2	2	2
3	3	3	–	2	2
4	4	3	4	–	1
5	5	5	5	5	–

A^4

	1	2	3	4	5
1	0	3	-1	4	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

P^4

	1	2	3	4	5
1	1	–	4	2	1
2	4	–	4	2	1
3	4	3	–	2	1
4	4	3	4	–	1
5	4	3	4	5	–

A^5

	1	2	3	4	5
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

P^5

	1	2	3	4	5
1	1	3	4	5	1
2	4	–	4	2	1
3	4	3	–	2	1
4	4	3	4	–	1
5	4	3	4	5	–

$$p^k(i,j) = p^{k-1}(k,j)$$

$A^{0'}$

	1	2	3	4	5	6
1	0	44	33	33	6	18
2	11	0	23	5	20	12
3	16	11	0	7	41	5
4	35	5	11	0	54	25
5	10	15	12	47	0	34
6	34	10	9	11	9	0

$P^{0'}$

	1	2	3	4	5	6
1	–	1	1	1	1	1
2	2	–	2	2	2	2
3	3	3	–	3	3	3
4	4	4	4	–	4	4
5	5	5	5	5	–	5
6	6	6	6	6	6	–

prev q/s

Src	dest	len	Path
1	2	1	1→5→4→3→2
1	3	-3	1→5→4→3
1	4	2	1→5→4
1	5	-4	1→5
2	1	3	2→4→1
2	3	-4	2→4→3
2	4	1	2→4
2	5	-1	2→4→5
3	1	7	3→2→4→1
3	2	4	3→2
3	4	5	3→2→4
3	5	3	3→2→4→5
4	1	2	4→1
4	2	-1	4→3→2
4	3	-5	4→3
4	5	-2	4→1→5
5	1	8	5→4→1
5	2	5	5→4→2
5	3	1	5→4→3
5	4	6	5→4

A^1

	1	2	3	4	5	6
1	0	44	33	33	6	18
2	11	0	23	5	17	12
3	16	11	0	7	22	5
4	35	5	11	0	41	25
5	10	15	12	43	0	28
6	34	10	9	11	9	0

P^1

	1	2	3	4	5	6
1	–	1	1	1	1	1
2	2	–	2	2	1	2
3	3	3	–	3	1	3
4	4	4	4	–	1	4
5	5	5	5	1	–	1
6	6	6	6	6	6	–

A^2

	1	2	3	4	5	6
1	0	44	33	33	6	18
2	11	0	23	5	17	12
3	16	11	0	7	22	5
4	15	5	11	0	22	17
5	10	15	12	20	0	27
6	21	10	9	11	9	0

P^2

	1	2	3	4	5	6
1	–	1	1	1	1	1
2	2	–	2	2	1	2
3	2	3	–	3	1	3
4	2	4	4	–	1	2
5	5	5	5	2	–	2
6	2	6	6	6	6	–

A3	1	2	3	4	5	6
1	0	44	33	33	6	18
2	11	0	23	5	17	12
3	16	11	0	7	22	5
4	16	5	11	0	22	16
5	10	15	12	19	0	17
6	21	10	9	11	9	0

P3	1	2	3	4	5	6
1	—	1	1	1	1	1
2	2	—	2	2	1	2
3	3	3	—	3	1	3
4	2	4	4	—	1	3
5	5	5	5	3	—	3
6	2	6	6	6	6	—

A4	1	2	3	4	5	6
1	0	38	33	33	6	18
2	11	0	16	5	17	12
3	16	11	0	7	22	5
4	16	5	11	0	22	16
5	10	15	12	19	0	17
6	21	10	9	11	9	0

P4	1	2	3	4	5	6
1	—	4	1	1	1	1
2	2	—	4	2	1	2
3	3	3	—	3	1	3
4	2	4	4	—	1	3
5	5	5	5	3	—	3
6	2	6	6	6	6	—

A5	1	2	3	4	5	6
1	0	21	18	25	6	18
2	11	0	16	5	17	12
3	16	11	0	7	22	5
4	16	5	11	0	22	16
5	10	15	12	19	0	17
6	19	10	9	11	9	0

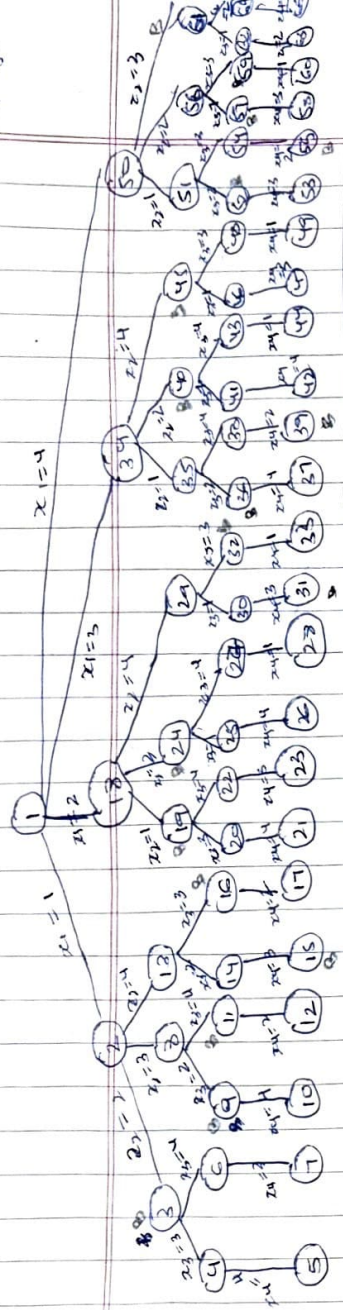
P5	1	2	3	4	5	6
1	—	5	5	3	1	1
2	2	—	4	2	1	2
3	3	3	—	3	1	3
4	2	4	4	—	1	3
5	5	5	5	3	—	3
6	5	6	6	6	6	—

A6	1	2	3	4	5	6
1	0	21	18	25	6	18
2	11	0	16	5	17	12
3	16	11	0	7	14	5
4	16	5	11	0	22	16
5	10	15	12	19	0	17
6	19	10	9	11	9	0

P6	1	2	3	4	5	6
1	—	5	5	3	1	1
2	2	—	4	2	1	2
3	3	3	—	3	1	3
4	2	4	4	—	1	3
5	5	5	5	3	—	3
6	5	6	6	6	6	—

Page 07/10/26

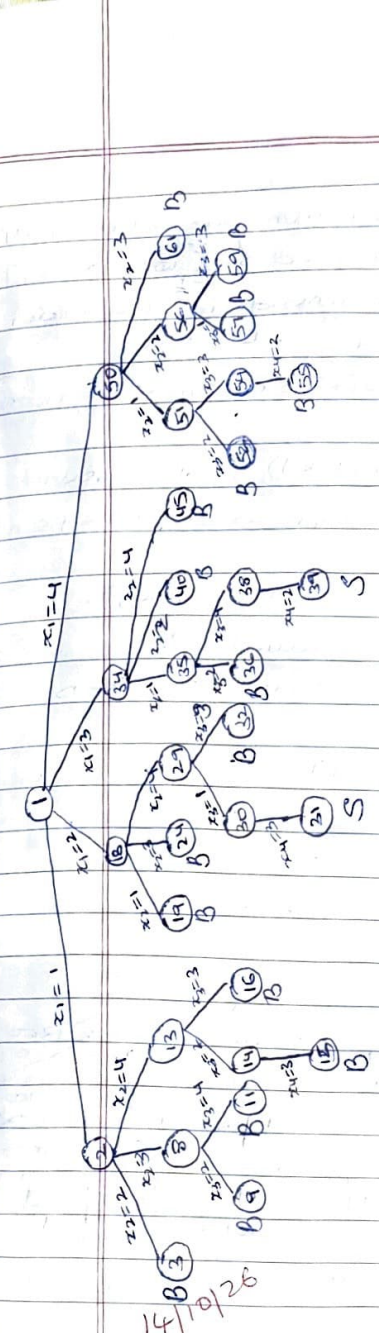
Src	dest	len	Path
1	2	21	1→5→2
1	3	18	1→5→3
1	4	25	1→5→3→4
1	5	6	1→5
1	6	18	1→6
2	1	11	2→1
2	3	16	2→4→3
2	4	5	2→4
2	5	17	2→1→5
2	6	12	2→6
3	1	16	3→1
3	2	11	3→2
3	4	7	3→4
3	5	14	3→6→5
3	6	5	3→6
4	1	16	4→2→1
4	2	5	4→2
4	3	11	4→3
4	5	22	4→2→1→5
4	6	16	4→3→6
5	1	10	5→1
5	2	15	5→2
5	3	12	5→3
5	4	19	5→3→4
5	6	17	5→3→6
6	1	19	6→5→1
6	2	10	6→2
6	3	9	6→3
6	4	11	6→4
6	5	9	6→5



k - queen
r - column

```
Algorithm Place(k, i)
// Returns true if a queen can be placed in kth row k
// pth column. Otherwise it returns false.
// x[] is a global array where first k-1 values
// have been set. Abs(r) returns absolute value
// of r.
{ for j := 1 to k-1 do
    if ((x[j] = i) or (Abs(j-k) = Abs(x[j] - i)))
      return false;
    else
      return true;
}

Algorithm NQueen(k, n)
// Using backtracking this procedure prints all
// possible placements of n queen on a n×n
// chessboard so that they are nonattacking
{ for i := 1 to n do
  { if (Place(k, i)) then
    { x[k] := i;
      if (k = n) then
        write(x[1:n]);
      else NQueen(k+1, n);
    }
  }
}
```

Solution vector

(2,4,1,3) X $x_1=2$ $x_2=4$ $x_3=1$ $x_4=3$

(3,1,4,2) X $x_1=3$ $x_2=1$ $x_3=4$ $x_4=2$

Matrix

	1	2	3	4
1	0	1	0	0
2	0	0	0	1
3	1	0	0	0
4	0	0	1	0

	1	2	3	4
1	0	0	1	0
2	1	0	0	0
3	0	0	0	1
4	0	1	0	0

14/10/26

14/10/26